

79. Le même jeu, deux fois

Une balle à la suite, trois balles par méthodes — le corps est le même.

1. Ce qu'il faut comprendre

À la suite, une balle demande quatre variables et huit lignes dans `main` : `if (versDroite) { x += 2; if (x > 150) versDroite = 0; }`, et ainsi de suite. Pour **une** balle, c'est parfait — 1427 octets.

Ci-dessous, trois balles **par méthodes**. Le corps de `avancer()` est **mot pour mot** celui de la version à la suite ; ce qui a changé, c'est qu'il est écrit une fois pour trois, et rangé avec les données sur lesquelles il travaille. 1956 octets, et quinze octets de mémoire.

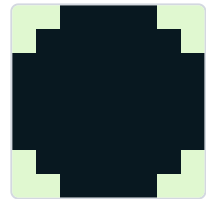
La seule règle honnête : écrire à la suite tant que c'est un objet, nommer par une fonction quand le geste se répète, ranger dans une méthode quand les gestes et les données vont manifestement ensemble. Et ne pas choisir d'avance — recompiler, et lire le compte rendu.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1985 octets** de cartouche.

1. UN DESSIN — BALLE

```
Tuile BALLE = {
    "..####..", ".#####.", "#####", "#####",
    "#####", "#####", ".#####.", "..####..",
};
```



à l'écran

2. UN ENREGISTREMENT — BALLE

```
struct Balle {
    uint8_t x, y, versDroite, versBas, pas;

    void avancer() {
        if (versDroite) { x += pas; if (x > 150) versDroite = 0; }
        else { x -= pas; if (x < 10) versDroite = 1; }

        if (versBas) { y += pas; if (y > 130) versBas = 0; }
        else { y -= pas; if (y < 10) versBas = 1; }
    }
};
```

3. UN TABLEAU — BALLES

```
Balle balles[3];
```

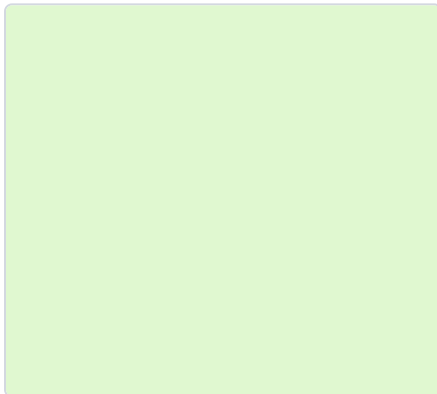
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    for (uint8_t i = 0; i < 3; i++) {
        balles[i].x = 20 + i * 40;
        balles[i].y = 30 + i * 20;
        balles[i].versDroite = i % 2;
        balles[i].versBas = 1;
        balles[i].pas = 1 + i;
    }

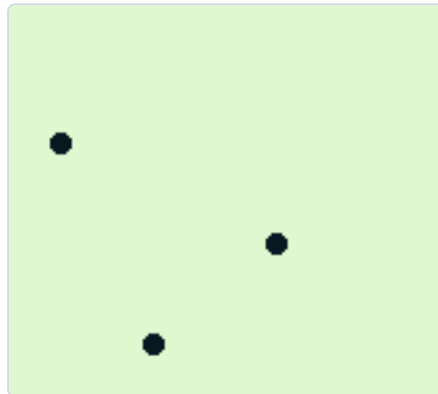
    while (true) {
        for (uint8_t i = 0; i < 3; i++) {
            balles[i].avancer();
            sprite(i, balles[i].x, balles[i].y, BALLE);
        }
        image();
    }
}
```

3. Ce qui se passe, étape par étape

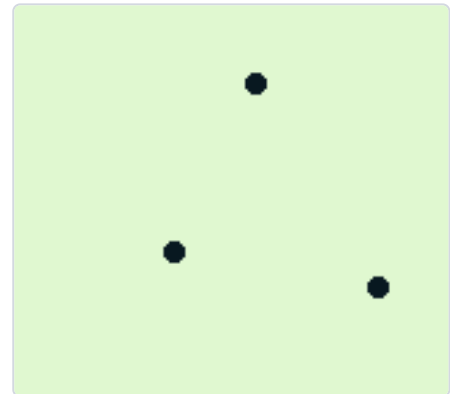
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



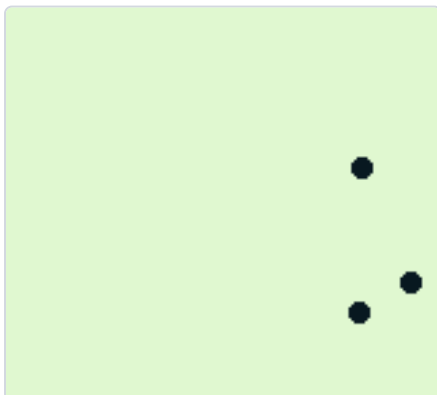
la toute première image — la console vient de commencer — image 1



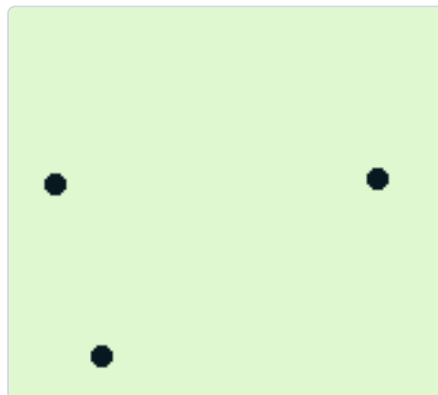
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Trois balles qui rebondissent, à trois vitesses différentes.

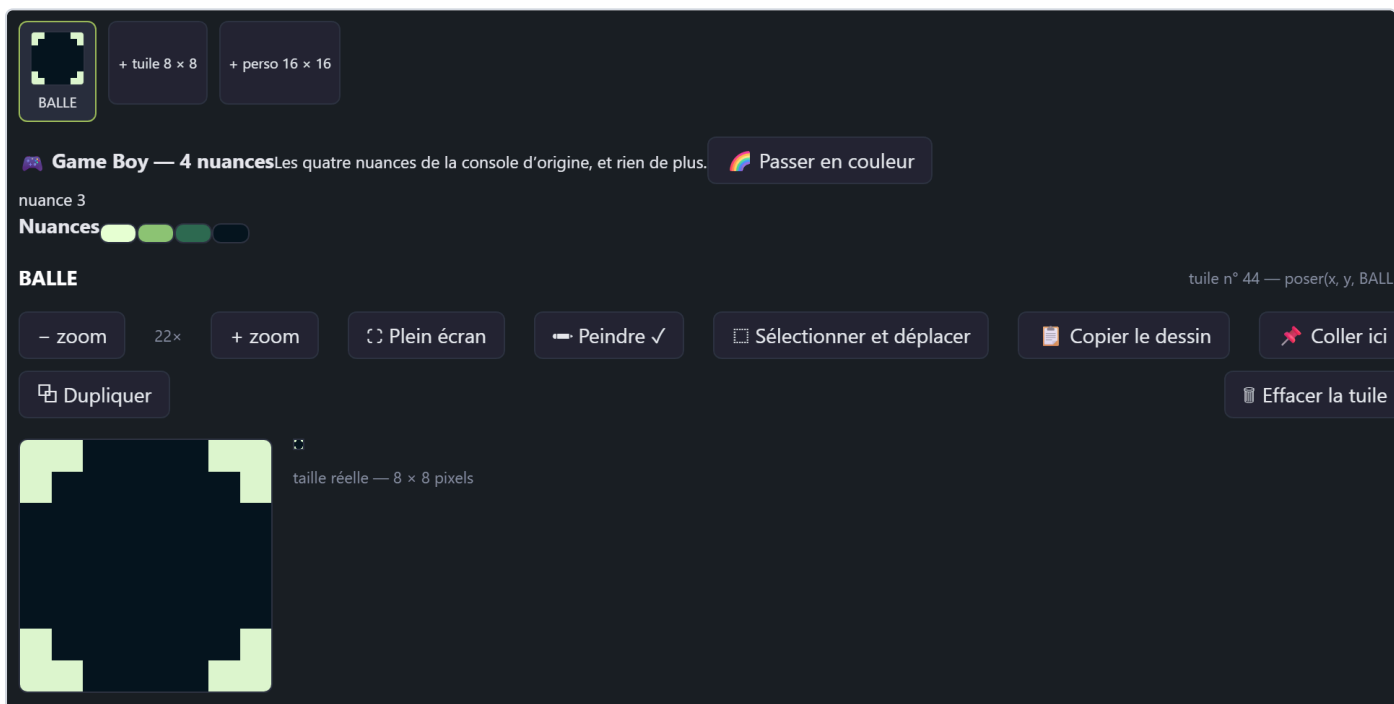
5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les trois balles bougent
- ✓ elles ne vont pas à la même vitesse
- ✓ aucune ne sort du cadre en trois cents images (0 écarts)
- ✓ la méthode sert les trois

6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.



Ensuite : **80. Ce que le compilateur refuse, et pourquoi** — niveau 10 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++